AthenaK M1 neutrino transport and BNS_nurates

This document collects the workflow of the M1 neutrino transport implementation coupled to the BNS_nurates opacity library.

M1 neutrino transport

PRELIMINARIES The M1 neutrino transport scheme in `AthenaK` is based on the implementation in `THC_M1` [1]. At it's heart, the M1 neutrino transport scheme tries to solve the 7-dimensional Boltzmann equation in general relativistic formulation, i.e.

$$p^\alpha \frac{\partial f}{\partial x^\alpha} - \Gamma^\alpha_{\beta\gamma} p^\beta p^\gamma \frac{\partial f}{\partial p^\alpha} = \mathcal{C}[f], \tag{1}$$

where $f_\nu(x^\alpha, p^i)$ is neutrino distribution function. Due to it's dimensionality, we choose a reduction to 3+1 dimensions, by performing a angular truncation over Ω. The first two angular moments of f_ν, i.e. the energy density and flux, are kept and the system is closed with a prescription for the second moment, i.e. the pressure. Integration is performed over the neutrino energy ε, so all the spectral information is gone from the evolved state. $\mathcal{C}[f]$ is the collision integral, appearing in the Boltzmann equation.
The M1 scheme requires a formulation of the radiation stress-energy tensor $T^{\mu\nu}_{(\nu)}$ in the Eulerian (normal observer $n^\mu = \frac{1}{\alpha}(1, -\beta^i)$) and the fluid frame (comoving observer u^μ) for each neutrino species (ν), i.e.

$$\text{Eulerian frame:} \quad T^{\mu\nu}_{(\nu)} = E_{(\nu)} n^\mu n^\nu + F^\mu_{(\nu)} n^\nu + n^\mu F^\nu_{(\nu)} + P^{\mu\nu}_{(\nu)}, \; F^\mu_{(\nu)} n_\mu = P^{\mu\nu}_{(\nu)} n_\nu = 0, \tag{2}$$

$$\text{Fluid frame:} \quad T^{\mu\nu}_{(\nu)} = J_{(\nu)} u^\mu u^\nu + H^\mu_{(\nu)} u^\nu + u^\mu H^\nu_{(\nu)} + K^{\mu\nu}_{(\nu)}, \; H^\mu_{(\nu)} u_\mu = K^{\mu\nu}_{(\nu)} u_\nu = 0, \tag{3}$$

where $E_{(\nu)}/J_{(\nu)}$, $F^\mu_{(\nu)}/H^\mu_{(\nu)}$, and $P^{\mu\nu}_{(\nu)}/K^{\mu\nu}_{(\nu)}$ are the radiation energy density, the radiation flux, and the radiation pressure tensor in the Eulerian/fluid frame. The most important operation in `AthenaK`'s M1 scheme is the calculation of the stress-energy tensor in the Eulerian frame using the evolved Eulerian variables $(E_{(\nu)}, F^\mu_{(\nu)})$ to compute the radiation pressure tensor $P^{\alpha\beta}_{(\nu)}$ and then assemble everything according to Equ. (2). Finally, the fluid frame variables, i.e. $(J^{(\nu)}, H^{(\nu)}_\mu)$, can be obtained by contracting with the fluid four-velocity

$$J^{(\nu)} = T^{(\nu)}_{\mu\nu} u^\mu u^\nu, \tag{4}$$

$$H^{(\nu)}_\mu = -{}^\perp h^\nu_\mu u^\rho T^{(\nu)}_{\nu\rho}, \tag{5}$$

where ${}^\perp h^\nu_\mu = \delta^\nu_\mu + u^\nu u_\mu$ is the fluid projector.
As mentioned above, the evolved variables in `AthenaK`'s M1 implementation are the radiation energy density $E_{(\nu)}$, the radiation flux $F^\mu_{(\nu)}$, and also the neutrino number current $N^\alpha_{(\nu)}$ for each neutrino species, i.e. electron neutrinos ν_e, anti-electron neutrinos $\bar{\nu}_e$, heavy lepton neutrinos ν_x, and anti-heavy lepton neutrinos $\bar{\nu}_x$. The evolution of the neutrino number current is desired, because weak reactions can alter the electron fraction of the matter, while conserving the total lepton number of the system. In total, we find a system of three evolution equations

$$\partial_t \tilde{E} + \partial_j(\alpha \tilde{F}^j - \beta^j \tilde{E}) = \alpha \tilde{P}^{ij} K_{ij} - \tilde{F}_j \gamma^{jk} \partial_k \alpha - \underbrace{\alpha n_\mu \tilde{S}^\mu}_{\text{matter}}, \tag{6}$$

$$\partial_t \tilde{F}_i + \partial_j(\alpha \tilde{P}^j_i - \beta^j \tilde{F}_i) = -\tilde{E} \partial_i \alpha + \tilde{F}_k \partial_i \beta^k + \frac{\alpha}{2} \tilde{P}^{jk} \partial_i \gamma_{jk} + \underbrace{\alpha \gamma^\mu_i \tilde{S}_\mu}_{\text{matter}}, \tag{7}$$

$$\partial_t \tilde{N} + \partial_j(\alpha n_\nu \tilde{f}^j_\nu) = \alpha(\tilde{\eta}_0 - \tilde{\kappa}_{0,a} n_\nu), \tag{8}$$

where $\tilde{A} = \sqrt{\gamma} A$ are densitized variables (in the case of the radiation pressure tensor this is implicit by construction from the densitized M1 variables), and $\eta_0/\kappa_{0,a}$ are the neutrino number emission/absorption coefficients. S^μ is the term incorporating the interaction between the neutrino radiation and the fluid, given by

$$S^\mu = (\eta - \kappa_a J) u^\mu - (\kappa_a + \kappa_s) H^\mu, \tag{9}$$

where η is the neutrino emissivity, κ_a the absorption, and κ_s the scattering coefficient. The combination $\eta - \kappa_a J$ drives the neutrino energy density towards a black-body-like equilibrium J_{eq}, while scattering only damps H^μ, i.e. the radiation

flux.
The final missing piece to the above machinery is the closure, which gives the radiation pressure tensor P^{ij} in terms of (E, F_i). The Minerbo closure mostly used in `AthenaK` defines

$$P^{ij} = \frac{3\chi - 1}{2} P^{ij}_{\rm thin} + \frac{3(1-\chi)}{2} P^{ij}_{\rm thick}, \tag{10}$$

where $\chi \in [1/3, 1]$ is the Eddington-factor and which is a function of the fluid-frame flux factor $\xi = \frac{\sqrt{H_\mu H^\mu}}{J}$, i.e.

$$\chi(\xi) = \frac{1}{3} + \xi^2 \left(\frac{6 - 2\xi + 6\xi^2}{15} \right). \tag{11}$$

The problem here is, that ξ depends on H^μ, which depends on P^{ij}, which depends on $\chi(\xi)$. This implicit chain can be solved by a root-finding algorithm of

$$f(\xi) = J(\xi)^2 \xi^2 - H_\mu(\xi) H^\mu(\xi) = 0. \tag{12}$$

In optically thick regions we find $H_\mu \sim 0$ and $\chi \sim \frac{1}{3}$, so $P^{ij} \sim P^{ij}_{\rm thick}$. Conversely, in optically thin regions $\chi \sim 1$, so $P^{ij} \sim P^{ij}_{\rm thin}$. Both regimes define the radiation pressure tensor as

$$P^{\rm thin}_{\alpha\beta} = \frac{E}{F_\mu F^\mu} F_\alpha F_\beta, \tag{13}$$

$$P^{\rm thick}_{\alpha\beta} = \left(\frac{2W^2 - 1}{2W^2 + 1} E - 2W^2 F^\mu v_\mu \right) (4W^2 v_\alpha v_\beta + g_{\alpha\beta} + n_\alpha n_\beta) + W(\gamma^\kappa_\alpha H_\kappa v_\beta + \gamma^\kappa_\beta H_\kappa v_\alpha), \tag{14}$$

where

$$\gamma^\kappa_\alpha H_\kappa = \frac{F_\alpha}{W} + \frac{W}{2W^2 + 1} v_\alpha ((4W^2 + 1) F^\mu v_\mu - 4W^2 E). \tag{15}$$

Lastly, instead of the Eulerian number density N, we require the one in the fluid frame n. Both are connected via the ratio

$$\Gamma = \frac{N}{n_\nu} = W \frac{E}{J} \left(1 - \frac{F_i v^i}{E} \right), \tag{16}$$

which is the generalized Lorentz factor of the radiation. Number is advected along the unit-normalized number-flux direction $f^\mu_\nu = u^\mu + H^\mu / J$. N for instance is then used to compute the mean neutrino energy $\langle \varepsilon \rangle = J / n_\nu$, which is the only spectral information the grey scheme has.

IMPLEMENTATION

radiation_m1.hpp: The M1 implementation in `AthenaK` defines a 5D array for the evolved variables (E, F^i, N) `u0`, a similar array for the evolved variables at intermediate RK steps `u1` and on a two times coarser grid `coarse_u0`. The eddington factor χ is also stored inside a separate 5D array `chi`. A 5D array for η_0 (`eta_0`) and $\kappa_{0,a}$ (`abs_0`), i.e. the neutrino number emissivity/absorptivity coefficients that enter the evolution equation for the neutrino number current in Equ. (8) is also set. Lastly, the neutrino emissivity coefficient η (`eta_1`), the absorption coefficient κ_a (`abs_1`), and the neutrino scattering coefficient κ_s (`scat_1`) which enter the matter-radiation coupling source term in Equ. (9) are also stored in separate arrays. Note: the fluxes of the evolved quantities on cell faces are also stored in a separate 5D array `uflx`. For more information, see `radiation_m1.hpp`. This header defines all the M1 tasks, communication buffers, container holding the task ID's, and some widely-used variables such as the number of evolved variables per species, the number of species etc.
radiation_m1_params.hpp: In this file, there are set of enumerators and structs to store the parameters of a specific M1 setup:

- `RadiationM1Closure`: defines enumerators for the M1 closure, i.e. `Minerbo`, `Eddington`, `Thin`, and `Kershaw`
- `RadiationM1OpacityType`: defines enumerators for the neutrino opacity library used, i.e. `None`, `Toy`, `BnsNurates`, and `Photons`

- `RadiationM1SrcUpdate`: defines enumerators for the method to treat radiation sources, i.e. `Explicit`, and `Implicit`
- `RadiationM1ClosureSolver`: defines enumerators for the root-finder used to determine the closure (Minerbo-only), i.e. `ClosureBrent` (safeguarded Brent-Dekker; default), and `ClosureNewton` (pure Newton-Raphson)
- `RadiationM1Params`: defines a struct that holds all the necessary M1 parameters

Parameter	Description
`closure_type` (RadiationM1Closure)	Choice of the closure (see above)
`closure_solver` (RadiationM1ClosureSolver)	Choice of the root-finder for the solver (see above)
`opacity_type` (RadiationM1OpacityType)	Choice of the opacity library (see above)
`src_update` (RadiationM1SrcUpdate)	Choice of the source update (see above)
`gr_sources` (bool)	Whether to include GR sources
`matter_sources` (bool)	Whether to include matter sources
`theta_limiter` (bool)	Whether to activate the Θ flux limiter (minmod)
`beam_sources` (bool)	Whether to include beam sources
`backreact` (bool)	Whether to include the matter-radiation backreaction source
`backreact_tmunu` (bool)	Whether to include the radiation contribution to the stress-energy tensor
`nspecies` (int)	Number of neutrino species (currently 4)
`closure_epsilon` (Real)	Precision with which to find the closure
`closure_maxiter` (int)	Maximum number of iterations in closure root finder
`closure_newton_bracket` (bool)	Safe-guard for the Newton closure solver
`minmod_theta` (Real)	Value of Θ for the minmod limiter
`rad_E_floor` (Real)	Radiation energy density E floor
`rad_N_floor` (Real)	Radiation number density N floor
`rad_eps` (Real)	Value for imposing $F_a F^a < (1 - rad\_eps)E^2$
`source_Ye_max` (Real)	Maximum allowed electron fraction Y_e for matter
`source_Ye_min` (Real)	Minimum allowed electron fraction Y_e for matter
`source_limiter` (Real)	Limiter for matter source (used to avoid negative energies)
`source_epsabs` (Real)	Target absolute precision for non-linear solver
`source_epsrel` (Real)	Target relative precision for non-linear solver
`source_maxiter` (int)	Maximum number of iterations for non-linear solver
`source_thick_limit` (Real)	Use optically thick limit, if equilibration time larger than timestep over this factor
`source_therm_limit` (Real)	Assume neutrinos to be thermalized above this optical depth
`source_scat_limit` (Real)	Use this scattering limit, if isotropization time less than timestep over this factor

- `SrcSignal`: defines enumerators for the source signal, i.e. `SrcThin`, `SrcEquil`, `SrcScat`, `SrcOk`, `SrcEddington`, and `SrcFail`
- `RadiationM1Beam`: defines a struct holding parameters for radiation beams, i.e. `beam_source_vals` (1D array), `beam_ymin`, and `beam_ymax`
- `SrcParams`: defines a struct with variables and tensors used for the computation of the source terms

radiation_m1.cpp: Complementing the `radiation_m1.hpp` header, the corresponding `radiation_m1.cpp` defines the M1 constructor/destructor and sets all the necessary parameters that exist in the module. Note: the block `<adm>` is needed to run with M1, because supported are only dynamical spacetimes. The default values of `RadiationM1Params` (defined as `params` in `radiation_m1.hpp`) are:

Parameter	Parameter name in parameter file	Default value
params.closure_type (RadiationM1Closure)	<radiation_m1> closure_fun	"minerbo" (Minerbo) (choices: "eddington" (Eddington), "thin" (Thin), and "kershaw" (Kershaw))
params.closure_solver (RadiationM1ClosureSolver)	<radiation_m1> closure_solver	"brent" (ClosureBrent) (choices: "newton" (ClosureNewton))
params.opacity_type (RadiationM1OpacityType)	<radiation_m1> opacity_type	"none" (choices: "bns-nurates" (BnsNurates), "toy" (Toy), "photons" (Photons))
params.src_update (RadiationM1SrcUpdate)	<radiation_m1> src_update	"implicit" (Implicit) (choices: "explicit" (Explicit))
params.gr_sources (bool)	<radiation_m1> gr_sources	true
params.matter_sources (bool)	<radiation_m1> matter_sources	true
params.theta_limiter (bool)	<radiation_m1> theta_limiter	false (recommended: true)
params.beam_sources (bool)	<radiation_m1> beam_sources	false
params.backreact (bool)	<radiation_m1> backreact	true
params.backreact_tmunu (bool)	<radiation_m1> backreact_tmunu	true
params.nspecies (int)	-	M1_TOTAL_NUM_SPECIES (set to 4 if compiled with -DAthena_ENABLE_NURATES=ON in CMakeLists.txt, otherwise 1)
params.closure_epsilon (Real)	<radiation_m1> closure_epsilon	10^{-5}
params.closure_maxiter (int)	<radiation_m1> closure_maxiter	64
params.closure_newton_bracket (bool)	<radiation_m1> closure_newton_bracket	true (only used if the RadiationM1ClosureSolver is ClosureNewton)
params.minmod_theta (Real)	<radiation_m1> minmod_theta	1.0
params.rad_E_floor (Real)	<radiation_m1> rad_E_floor	10^{-30}
params.rad_N_floor (Real)	<radiation_m1> rad_N_floor	10^{-77}
params.rad_eps (Real)	<radiation_m1> rad_eps	10^{-14}
params.source_Ye_max (Real)	<radiation_m1> source_Ye_max	0.6
params.source_Ye_min (Real)	<radiation_m1> source_Ye_min	0.0
params.source_limiter (Real)	<radiation_m1> source_limiter	0.5
params.source_epsabs (Real)	<radiation_m1> source_epsabs	10^{-15}
params.source_epsrel (Real)	<radiation_m1> source_epsrel	10^{-5}
params.source_maxiter (int)	<radiation_m1> source_maxiter	64
params.source_thick_limit (Real)	<radiation_m1> source_thick_limit	20.0
params.source_therm_limit (Real)	<radiation_m1> source_therm_limit	-1.0
params.source_scat_limit (Real)	<radiation_m1> source_scat_limit	-1.0

In our case, we are only interested in the connection of M1 with BNS_nurates. If the opacity library is chosen to be BNS_nurates in the M1 constructor, then a set of separate library specific parameters are set. The specific struct is defined in radiation_m1_nurates.hpp as NuratesParams and gets stored in a nurates_params variable if the code is compiled with -DAthena_ENABLE_NURATES=ON. For completeness, we also list the parameters of this struct and the default values (defined in radiation_m1.cpp) here:

Nurates parameter	Parameter name in parameter file	Description (Default value)
nurates_params.opacity_tau_trap (Real)	<bns_nurates> opacity_tau_trap	Include effects of neutrino trapping above this optical depth (1.0)
nurates_params.opacity_tau_delta (Real)	<bns_nurates> opacity_tau_delta	Range of optical depths, over which trapping is introduced (1.0)
nurates_params.opacity_corr_fac_max (Real)	<bns_nurates> opacity_corr_fac_max	Maximum correction factor for optically thin regime (3.0)

nurates_params.nb_min (Real)	<bns_nurates> nb_min_fm-3	Minimum baryon number density in BNS_nurates (0.0)
nurates_params.temp_min_mev (Real)	<bns_nurates> temp_min_mev	Minimum temperature in BNS_nurates (0.0)
nurates_params.use_abs_em (bool)	<bns_nurates> use_abs_em	Enable β-processes in BNS_nurates (true)
nurates_params.use_pair (bool)	<bns_nurates> use_pair	Enable electron-positron annihilation in BNS_nurates (true)
nurates_params.use_brem (bool)	<bns_nurates> use_brem	Enable nucleon-nucleon bremsstrahlung in BNS_nurates (true)
nurates_params.use_iso (bool)	<bns_nurates> use_iso	Enable isoenergetic scattering of nucleons in BNS_nurates (true)
nurates_params.use_inelastic_scatt (bool)	<bns_nurates> use_inelastic_scatt	Enable inelastic scattering off electrons/positrons in BNS_nurates (false)
nurates_params.use_WM_ab (bool)	<bns_nurates> use_WM_ab	Flag for weak magnetism correction on absorption rates (true)
nurates_params.use_WM_sc (bool)	<bns_nurates> use_WM_sc	Flag for weak magnetism correction on scattering rates (true)
nurates_params.use_dU (bool)	<bns_nurates> use_dU	Flag for the correction $\Delta U = U_n - U_p$, i.e. difference of interaction potential of neutron in dense nuclear matter compared to interaction potential of proton (true)
nurates_params.use_dm_eff (bool)	<bns_nurates> use_dm_eff	Flag for treating the difference of the proton and neutron mass following a RMF approach (true)
np.use_equilibrium_distribution (bool)	<bns_nurates> use_equilibrium_distribution	Flag for using a equilibrium distribution for the distribution function (true)
nurates_params.use_kirchhoff_law (bool)	<bns_nurates> use_kirchhoff_law	Flag for using the kirchhoff law (true)
np.use_nonthermal_separated (bool)	<bns_nurates> use_nonthermal_separated	Whether to treat inelastic scattering separately from thermal processes (true)
nurates_params.use_NN_medium_corr (bool)	<bns_nurates> use_NN_medium_corr	Flag for inclusion of medium correction to HR98 NN bremsstrahlung kernel as in Fischer16 (true)
nurates_params.neglect_blocking (bool)	<bns_nurates> use_neglect_blocking	Flag for neglecting the blocking factor of antineutrino in pair processes (false)
nurates_params.use_decay (bool)	<bns_nurates> use_decay	Flag for inclusion of nucleon decay rates (false)
nurates_params.use_BRT_brem (bool)	<bns_nurates> use_BRT_brem	Flag for using the BRT06 bremsstrahlung implementation; default is HR98 (false)
nurates_params.eq_warmup_cycles (int)	<bns_nurates> eq_warmup_cycles	Number of equilibrium warm-up cycles for reconstruction (1)

In the constructor, after all the parameters are set, we allocate the necessary memory based on the grid setup for the defined arrays.

radiation_m1_closure.hpp: This header file defines a function called closure_fun, which returns the closure parameter $\chi(\xi)$

based on the specified closure. In particular:

$$\text{Minerbo: } \chi = \frac{1}{3} + \xi^2 \frac{6 - 2\xi + 6\xi^2}{15}, \tag{17}$$

$$\text{Thin: } \chi = 1.0, \tag{18}$$

$$\text{Eddington: } \chi = \frac{1}{3}, \tag{19}$$

$$\text{Kershaw: } \chi = \frac{1}{3} + \frac{2}{3}\xi^2. \tag{20}$$

This function is used to compute the closure in `radiation_m1_calc_closure.hpp` and in the root finding algorithm for ξ in `radiation_m1_roots_fns.hpp`.

radiation_m1_fermi.hpp: In this header file, approximate Fermi integrals after [2]. In particular:

$$f^{2p}(\eta) = \frac{1/3 + 3.2899/\eta^2}{1 - \exp(-1.8246/\eta)}, \tag{21}$$

$$f^{2m}(\eta) = \frac{2}{1 + 0.1092\exp(0.8908\eta)}, \tag{22}$$

$$f^2_{\eta>1e-3}(\eta) = \eta^3 f^{2p}(\eta), \tag{23}$$

$$f^2_{\eta<1e-3}(\eta) = \exp(\eta) f^{2m}(\eta), \tag{24}$$

$$f^{3p}(\eta) = \frac{0.25 + 4.9348/\eta^2 + 11.3644/\eta^4}{1 + \exp(-1.9039\eta)}, \tag{25}$$

$$f^{3m}(\eta) = \frac{6}{1 + 0.0559\exp(0.9069\eta)}, \tag{26}$$

$$f^3_{\eta>1e-3}(\eta) = \eta^4 f^{3p}(\eta), \tag{27}$$

$$f^3_{\eta<1e-3}(\eta) = \exp(\eta) f^{3m}(\eta). \tag{28}$$

Function $f^2(\eta)$ is used in `radiation_m1_nurates.hpp` to compute the number density of the neutrino species in $[\text{nm}^{-3}]$, and $f^3(\eta)$ is used in the same file to compute the energy density of the neutrino species in $[\text{MeV/nm}^3]$.

radiation_m1_helpers.hpp: In this header file a lot of useful helper functions are defined which are frequently used in the computation of the closure:

- `minmod2`: compute a double minmod, i.e. $\min(1, \min(\Theta r_l, \Theta r_p))$ which is used in the computation of the fluxes in `radiation_m1_fluxes` and uses `params.minmod_theta` (Θ)
- `CombinedIdx`: returns a combined index given a flavor index, variable index and the total number of variables, i.e. $I = i_{\text{var}} + i_{\text{flavor}} n_{\text{vars}}$, where i_{var} is the variable index, i_{flavor} is the flavor index and n_{vars} is the total number of variables (defined in `radiation_m1.hpp`)
- `calc_proj`: compute the fluid projector in the Eulerian frame ${}^{\perp}h^{\nu}_{\mu} = \delta^{\nu}_{\mu} + n^{\nu} n_{\mu}$ or in the fluid frame ${}^{\perp}h^{\nu}_{\mu} = \delta^{\nu}_{\mu} + u^{\nu} u_{\mu}$ which is used in the computation of the radiation flux in the fluid frame, i.e. it is used to project out the stress-energy tensor to obtain the radiation flux in the fluid frame
- `calc_K_from_rT`: projects out the radiation pressure tensor in any frame via $K_{\mu\nu} = {}^{\perp}h^{\kappa}_{\mu} {}^{\perp}h^{\rho}_{\nu} T_{\kappa\rho}$ (not used anywhere in the code)
- `calc_Pthin`: computes the radiation pressure tensor in the thin limit in the Eulerian frame via (see Equ. (13)), which is used when computing the closure
- `calc_Pthick`: computes the radiation pressure tensor in the thick limit in the Eulerian frame via (see Equ. (14)), which is used when computing the closure
- `flux_factor`: computes the flux factor $\xi^2 = \frac{H_\mu H^\mu}{J^2}$ (Note: this is not used anywhere in the code, only when computing some derived variables for the tracers, where we use ξ, i.e. the square-root of this function)
- `assemble_fnu`: assembles the unit-norm radiation number current as $f^{\mu}_{(\nu)} = u^{\mu} + \frac{H^{\mu}}{J}$, where the second term is dropped if J is below `rad_E_floor` (it is used for the computation of the fluxes in `radiation_m1_fluxes.cpp`)

- `compute_Gamma`: computes the ratio of the neutrino number densities in the Eulerian and the fluid frame, i.e. $\Gamma = \frac{WE}{J}\left(1 - \min\left((F_i v^i)/E, 1 - \texttt{params.rad\_eps}\right)\right)$ (used when computing the closure)
- `assemble_rT`: assembles the radiation stress-energy tensor in any frame via Equs. (2)-(3) (used when computing the closure)
- `calc_J_from_rT`: projects out the radiation energy in any frame via $E = T_{\mu\nu} n^\mu n^\nu$ or $J = T_{\mu\nu} u^\mu u^\nu$ (used when computing the closure)
- `calc_H_from_rT`: projects out the radiation fluxes in any frame via $F_\mu = -^\perp h^\nu_\mu n^\kappa T_{\nu\kappa}$ or $H_\mu = -^\perp h^\nu_\mu u^\kappa T_{\nu\kappa}$
- `apply_closure`: computes the closure in the Eulerian frame from the Eddington factor χ via Equ. (10)
- `calc_E_flux`: computes the radiation energy flux as $\mathbf{F}^i = \alpha F^i - \beta^i E$, which enters the evolution equation of the radiation energy (see Equ. (6))
- `calc_F_flux`: computes the flux of neutrino energy flux as $\mathbf{F}^i = \alpha P^i_k - \beta^i F_k$, which enters the evolution equation for the radiation flux (see Equ. (7)); Note: the variables given in the equation are densitized as they are in the code, but here we drop the tilde
- `calc_rad_sources`: computes the matter-radiation source via Equ. (9) (WARNING: be careful with densitization of the variables; used in `radiation_m1_sources.hpp` and `radiation_m1_update.cpp`)
- `calc_rE_source`: computes the source term for E, i.e. $\mathbf{S} = -\alpha S^\mu n_\mu$ (see Equ. (6); again be careful with densitization)
- `calc_rF_source`: computes the source term for F_μ, i.e. $\mathbf{S}_k = \alpha S^\mu \gamma_{k\mu}$ (see Equ. (7); again be careful with densitization)
- `apply_floor`: enforces that $E >$ `rad_E_floor` and $F_\mu F^\mu < (1 -$ `rad_eps`$)E^2$

radiation_m1_macro.hpp: This header defines macros for the evolution variables, i.e.

```
#define M1_E_IDX 0
#define M1_FX_IDX 1
#define M1_FY_IDX 2
#define M1_FZ_IDX 3
#define M1_N_IDX 4
```

radiation_m1_tensors.hpp: This header defines important functions for tensor operations and preparing tensors in general:

- `tensor_dot`: there are different instances of this function with separate function signatures; they compute $g^{\mu\nu} M_\mu G_\nu$ for some four-vectors M^μ and G^ν, $g^{ij} M_i G_j$ (spatial), they also compute $P^{\mu\nu} K_{\mu\nu} = g^{\mu\kappa} g^{\nu\rho} P_{\kappa\rho} K_{\mu\nu}$, the Euclidean dot product $M^\mu G_\mu$ for vectors and for tensors $M^{\mu\nu} G_{\mu\nu}$ (these functions are extensively used)
- `tensor_contract`: there are two instances of this function; the first defines the operation $F_\mu = g_{\mu\nu} F^\nu$ or the inverse, while the second defines $P^\mu_\nu = g^{\mu\kappa} P_{\kappa\nu}$ (these functions are extensively used)
- `tensor_contract2`: computes $P^{ij} = g^{ik} g^{jl} P_{kl}$ (this is used once in `radiation_m1_update.cpp`)
- `pack_n_d`: populates the normal vector $n_\mu = (-\alpha, 0, 0, 0)$
- `pack_beta_u`: populates the shift vector $\beta^\mu = (0, \beta^i)$
- `pack_u_u`: populates the four-velocity u^μ as $(W/\alpha, \tilde{u}^i - W\beta^i/\alpha)$
- `pack_v_u`: populates $v^\mu = (0, u^i/W + \beta^i/\alpha)$, where u^i are packed as the spatial components of `pack_u_u`
- `pack_F_d`: populates $F_\mu = (\beta^i F_i, F_i)$
- `pack_F_d`: populates F_i
- `pack_P_dd`: populates $P_{\mu\nu}$ (not used in the code)
- `get_w_lorentz`: computes the Lorentz factor from the fluid velocity via $W = \sqrt{1 + g_{\mu\nu} \tilde{u}^\mu \tilde{u}^\nu}$, where $\tilde{u}^\mu = (0, W v^i)$ is the Lorentz-weighted velocity that AthenaK uses

radiation_m1_linalg.hpp: This header defines a couple of linear algebra functions and a so-called `MathSignal` enumerator which has the fields `LinalgInvalid`, `LinalgEinval`, `LinalgSuccess`, `LinalgContinue`, `LinalgEnoprogj`, `LinalgEbadfunc`, and `LinalgEbadtol` which are primarily used for the root-finders:

- `norm_l2`: computes the L2-norm of a 4D vector, i.e. $||p||_2 = \sqrt{\sum_{i=0}^{4} p_i \cdot p_i}$, and of a 3D vector, i.e. $||p||_2 = \sqrt{\sum_{i=0}^{3} p_i \cdot p_i}$ (only used for other linalg operations)
- `dot`: computes the dot product of two 4D vectors, i.e. $d_i \cdot g_i$ (only used for other linalg operations)
- `sign`: returns the sign of a floating point number (only used for other linalg operations)
- `dscal`: multiplies the components of a 4D vector with a scalar, i.e. $a \cdot p_i$ (only used for other linalg operations)
- `householder_transform`: performs a transformation on a 4D vector p, i.e. it first computes the L2-norm $||x||_2$ of the components $i \neq 0$ (sub-vector x_i), then computes $\alpha = p_0$, $\beta = -\text{sign}(\alpha) \cdot \sqrt{\alpha^2 + ||x||_2^2}$, and $\tau = (\beta - \alpha)/\beta$. The latter is what the function returns, and the $i = 0$ component of the vector gets updated by checking $\alpha - \beta$ to be larger than a threshold `DBL_MIN`. If that is the case, then $p_0 = \beta$ and $p_i = s^{-1} x_i$, otherwise $p_0 = \beta$ and $p_i = \frac{\varepsilon}{s}\frac{1}{\varepsilon} x_i$, for some ε constant (not used in the code)
- `qr_factorize`: computes the QR decomposition of a square matrix. Given three matrices $\mathbf{A}$, $\mathbf{Q}$, and $\mathbf{R}$ it first sets all the components of $\mathbf{Q}$ and $\mathbf{R}$ to zero and fills a temporary matrix $\mathbf{V}$ with the values of $\mathbf{A}$. From this temporary matrix it takes each row as a sub-vector $\mathbf{V}_i$ and then iterates to fill the entries of this sub-vector with the values from the underlying matrix. The diagonal components of $\mathbf{R}$ get filled with $||V_i||_2$. The entries for the given row i of $\mathbf{Q}$ are filled via $V_i/\mathbf{R}_{ii}$ and each row gets the same values. Lastly, we iterate over the columns of $\mathbf{V}$ to retrieve a subvector V_k which is used to fill the off-diagnoal components of $\mathbf{R}$ via the dot product of Q_i and V_k. The off-diagonal components of $\mathbf{V}$ are computed as $\mathbf{V}_{ki} = \mathbf{V}_{ki} - \mathbf{R}_{ij}\mathbf{Q}_i$ (used for the hybrid root-solver)
- `givens_rotation`: generates a Givens rotation of $v = (x, y)$ to $(|v|, 0)$ (used for other linalg operations), i.e.

```
KOKKOS_INLINE_FUNCTION
void givens_rotation(const Real &a, const Real &b, Real &c, Real &s) {
  if (b == 0) {
    c = 1;
    s = 0;
  } else if (Kokkos::fabs(b) > Kokkos::fabs(a)) {
    Real t = -a / b;
    Real s1 = 1.0 / Kokkos::sqrt(1 + t * t);
    s = s1;
    c = s1 * t;
  } else {
    Real t = -b / a;
    Real c1 = 1.0 / Kokkos::sqrt(1. + t * t);
    c = c1;
    s = c1 * t;
  }
}
```

- `givens_apply_gv`: applies a rotation $v' = G^T v$ (used for other linalg operations)
- `givens_apply_qr`: applies a rotation $Q' = QG$ (used for other linalg operations)
- `qr_update`: updates a QR factorisation $QR = B$ to $B' = B + uv^T$ and used the givens operations above (used in the hybrid root-solver)
- `qr_R_solve`: computes the solution to the system $Rx = b$, where R is a right triangular matrix

radiation_m1_newdt.cpp: This file computes the minimum time-step based on the new grid setup for the M1 system after AMR.
radiation_m1_roots_brent.hpp: Implements the Brent-Dekker root-finder which uses a `BrentState` struct with fields $a,\ b,\ c,\ d,\ e,\ f_a,\ f_b,\ f_c$. The function `BrentInitialize` initializes the root-finder, i.e. it uses $x_l = 0.0$ and $x_h = 1.0$ in `radiation_m1_calc_closure.hpp`. In `BrentIterate` the main body of the iteration algorithm lives which gets complemented by `BrentTestInterval` to check whether the iteration should continue or if it is converged.
radiation_m1_roots_hybridsj.hpp: Provides functions for Powell's multiroot solver, i.e. for the Newton-Rapshon solver.
radiation_m1_roots_fns.hpp: Defines the `NewtonClosure` function for the Minerbo closure which solves Equ. (12) for ξ. Used in `radiation_m1_calc_closure.hpp` if `ClosureNewton` is selected by the user. A separate flag `closure_newton_bracket` sets a bracket $[x_{\rm lo}, x_{\rm hi}]$, such that the residual $f(\xi)$ is bracketed as $f(x_{\rm lo}) \leq 0 \leq f(x_{\rm hi})$ and the step is the Newton point, if it stays inside that bracket. This guarantees convergence to the physical root, because the Minerbo residual is bimodal and this can spoil the closure in the core.
radiation_m1_tmunu.cpp: This file adds the radiation contribution to the stress-energy tensor. It is guarded behind `backreact_tmunu`, which is on by default, so the algorithm executes. In essence, we follow the same procedure as elsewhere in the code to compute the radiation pressure tensor from the evolved M1 variables and then we fill `tmunu.E` (energy density) with the radiation energy density E, `tmunu.S_d` (momentum density) with radiation flux $\sqrt{\gamma}F_i$, and `tmunu.S_dd` (stress-energy tensor) with $T_{\mu\nu}$.
radiation_m1_fluxes.cpp: This file calculates the 3D fluxes for our grey M1 scheme. `CalcFlux` is a wrapper that computes the flux by computing the spacetime metric $g_{\mu\nu}$ and $g^{\mu\nu}$, the spatial metric γ^{ij}, the Lorentz factor and the fluid projector, and it extracts (E, F_i). With this it computes the radiation pressure tensor in the Eulerian frame $P_{\mu\nu}$ and computes it contraction P^{μ}_{ν} (`apply_closure`). The radiation pressure tensor is the last ingredient to assemble the radiation stress-energy tensor $T_{\mu\nu}$ via `assemble_rT`. Projecting out the stress-energy tensor to the fluid frame gains the energy density and the radiation flux, i.e. the pair (J, H_i) in the fluid frame. The unit-norm radiation number current $f^{\mu}_{(\nu)}$ (`assemble_fnu`) and the ratio between the neutrino number densities in the Eulerian and fluid frame (`compute_Gamma`) are calculated. This is used to compute the neutrino number density in the fluid frame $n_{(\nu)} = \frac{N}{\Gamma}$. Finally, with `calc_E_flux` and `calc_F_flux` the radiation energy flux and flux of neutrino energy flux are calculated. Another thing that is computed is the maximum value of the speed of light between the right and left cells via

$$c_{i+1/2} = \max_{a\in\{i,i+1\}} \{|\alpha_a \sqrt{\gamma^{xx}_a} \pm \beta^x_a|\}. \tag{29}$$

The flux of the neutrino number density is calculated via $\mathbf{F}^i = \alpha n_{(\nu)} f^i_{(\nu)}$. `CalcFlux` is the called inside `CalculateFluxes` to calculate the flux in each of the three spatial dimensions. In particular, the flux at the face $x_{i+1/2}$ is computed via

$$F_{i+1/2} = F^{\rm HO}_{i+1/2} - A_{i+1/2}\varphi_{i+1/2}\left(F^{\rm HO}_{i+1/2} - F^{\rm LO}_{i-1/2}\right), \tag{30}$$

which is a construction of a non-diffusive second-order flux $F^{\rm HO}$ and a diffusive first-order correction $F^{\rm LO}$, whcih are defined via

$$F^{\rm HO}_{i+1/2} = \frac{f(u_i) + f(u_{i+1})}{2}, \tag{31}$$

$$F^{\rm LO}_{i+1/2} = \frac{1}{2}[f(u_i) + f(u_{i+1})] - \frac{c_{i+1/2}}{2}[u_{i+1} - u_i]. \tag{32}$$

Above $\varphi_{i+1/2}$ is the so-called flux-limiter and $A_{i+1/2}$ is a coefficient to switch off the diffusive correction at high optical depth. The latter is determined based on

$$A_{i+1/2} = \min\left(1, \frac{1}{\Delta x \kappa_{\rm ave}}\right), \tag{33}$$

$$\kappa_{\rm ave} = \frac{1}{2}[(\kappa_a)_i + (\kappa_a)_{i+1} + (\kappa_s)_i + (\kappa_s)_{i+1}], \tag{34}$$

such that the coefficient is around one in optically thin regions while it is way less than one in optically thick regions. The flux-limiter is computed using a standard minmod approach (`minmod2` in helpers)

$$\varphi_{i+1/2} = \min\left[1, \min\left(\frac{u_i - u_{i-1}}{u_{i+1} - u_i}, \frac{u_{i+2} - u_{i+1}}{u_{i+1} - u_i}\right)\right]. \tag{35}$$

The above discretized fluxes at faces are used to compute the approximate derivative of the fluxes via $\partial_x f(u) \approx \frac{F_{i-1/2} + F_{i+1/2}}{\Delta x}$. To sum up, the above routine in `CalculateFluxes` for each direction computes the flux $f(u_i)$ and $f(u_{i+1})$, then computes

$\kappa_{\rm ave}$ used for $A_{i+1/2}$. Determining $\varphi_{i+1/2}$ and assembling everything using the definitions for the high-order and low-order fluxes yields $F_{i-1/2}$ and $F_{i+1/2}$. This routine is called in `radiation_m1_tasks.cpp`.
radiation_m1_calc_closure.hpp: Defines one Kokkos inline function called `calc_closure`, which is wrapper to `calc_closure` (defined in `radiation_m1_helpers.hpp`). Based on the specified closure, it determines χ where in the case of the Minerbo closure it uses either the Newton or the Brent-Decker rootfinder. If the initialization of the latter yields no root, then χ either is set to the lower limit of $\frac{1}{3}$ or to 1 depending on the outcome. Otherwise `BrentIterate` tries to determine the root and `closure_fun` (in `radiation_m1_closure.hpp`) computes χ based on the root ξ.
radiation_m1_calc_closure.cpp: Defines one routine called `FloorAndCalcClosure` (used in `dyn_grmhd.cpp`) which computes the closure χ using `calc_closure` from `radiation_m1_calc_closure.hpp` at every grid point, except the point is inside the radiation mask. The flag `is_chi_updated` is set to true if χ was determined.
radiation_m1_sources.hpp: This header file prepares a couple of functions for the sources (most of the source-specific parameters are stored in a struct of `SrcParams`):

- `prepare_closure`: Packs a vector F_μ with `pack_F_d` and contracts to get F^μ. Finally, the closure parameter is computed with `calc_closure`.
- `prepare_sources`: Assembles the stress-energy tensor $T_{\mu\nu}$ with `assemble_rT`, computes J with `calc_J_from_rT` and H_μ with `calc_H_from_rT` and stores both in a `SrcParams` struct called `src_params`. It applies the floors on J and H_μ and the calculates the radiation sources with `calc_rad_sources` (defined in `radiation_m1_helpers.hpp` and implements Equ. (9)). Using S_μ from the previous step, the sources $\mathbf{S}$ of E and F_μ are calculated.
- `prepare`: Calls `prepare_closure` and `prepare_sources` (used in the root-finders).
- `explicit_update`: Computes the explicit update $(E/F)_{\rm new} = (E/F)^* + c\Delta t \cdot t S_\mu [E/F]$ (only used in `radiation_m1_sources.hpp`).
- `source_update_ll`: Implements the main driver function to solve the implicit problem $q^{\rm new} = q^* + \Delta t S[q^{\rm new}]$, where the source term is defined by Equ. (9). In the non-stiff limit, i.e. $c\Delta t\kappa_a < 1$ and $c\Delta t\kappa_s < 1$, the function calls `prepare` to prepare the closure and the sources and then computes the explicit update via `explicit_update`. It then calls `prepare_closure` again on the new M1 variables. In the thick and scattering dominated limit it returns. Otherwise the multiroots solver is used to determine the solution to the implicit problem.
- `source_update`: Is a wrapper to `source_update_ll` and first computes the source update based on the specified solver and if the multiroots solver was not able to find a solution then `source_update_ll` is called again with the Eddington closure (called in `radiation_m1_update.cpp`).

radiation_m1_update.cpp: The `TimeUpdate` function is the main routine that performs the M1 time update. It calls a wrapper function `TimeUpdate_` based on the specified equation of state type (`NormalLogs` or `NQTLogs`, as well as the number ghost zones; if MHD and hydro are Null `NQTLogs` are used), which does most of the heavy lifting. The wrapper function performs updates of an implicit problem $F = F^* + c\Delta t S[F]$ where $F^* = F^k + c\Delta t A$ of the form:

$$F^m = F^k + \frac{\Delta t}{2}[A[F^k] + S[F^m]], \tag{36}$$

$$F^{k+1} = F^k + \Delta t[A[F^m] + S[F^{k+1}]]. \tag{37}$$

The computation in `TimeUpdate_` is as follows:

1. In excised cells, set the M1 variables to 0 and skip the flux divergence, geometric, and matter-source updates (as in THC).
2. A couple of GR-quantities are set up: Compute the spacetime-metric $g_{\mu\nu}$ and $g^{\mu\nu}$, as well as the spatial metric γ^{ij} plus fill the extrinsic curvature tensor K_{ij}. Pack a normal vector n_μ (using `pack_n_d`) and contract with `tensor_contract` to get n^μ. Both normal vectors are used to compute γ^μ_ν via $\gamma^\mu_\nu = \delta^\mu_\nu + n^\mu n_\nu$. Furthermore, we pack a shift vector β^μ via `pack_beta_u` and contract this with `tensor_contract` to get β_μ. Lastly, $\sqrt{\gamma}$ is computed using `adm::SpatialDet`.
3. Compute the derivatives of these GR-quantities, i.e. compute derivative of the lapse $\partial_i \alpha$, derivative of the shift $\partial_i \beta^j$, and the derivative of the spatial metric $\partial_k \gamma_{ij}$.
4. Compute the fluid quantities such as the Lorentz factor, the four-velocity u^μ (and its contraction), the three-velocity v^i (and its contraction), and the fluid projector ${}^\perp h^\mu_\nu$.

5. Compute the contribution from the flux and geometric sources: the first is computed as $\partial_j f(u) \approx \frac{F_{i-1/2}-F_{i+1/2}}{\Delta x_j}$, where the flux sources are

$$\mathbf{U} = \begin{pmatrix} \sqrt{\gamma} n \Gamma \\ \sqrt{\gamma} E \\ \sqrt{\gamma} F_k \end{pmatrix}, \tag{38}$$

$$\mathbf{F} = \begin{pmatrix} \alpha \sqrt{\gamma} n f^i \\ \sqrt{\gamma}[\alpha F^i - \beta^i E] \\ \sqrt{\gamma}[\alpha P^i_k - \beta^i F_k] \end{pmatrix}. \tag{39}$$

For the latter we extract the evolved M1 variables (E, F_i), apply the closure with `apply_closure` to get $P_{\mu\nu}$, get the spatial components of both the radiation flux and the radiation pressure tensor and then computed the geometric sources according to Equ. (6) for E and Equ. (7) for F_i (also see [1] for the full form of the sources). In full generality:

$$\mathbf{G} = \begin{pmatrix} 0 \\ \alpha \sqrt{\gamma}[P^{ik} K_{ik} - F^i \partial_i \log \alpha] \\ \sqrt{\gamma}[F_i \partial_k \beta^i - E \partial_k \alpha + \frac{\alpha}{2} P^{ij} \partial_k \gamma_{ji}] \end{pmatrix}. \tag{40}$$

6. Lastly, the contribution from the matter sources is computed: for that evolved variables are first extracted (E, F_i) and the closure $P_{\mu\nu}$ is calculated via `apply_closure`. The stress-energy tensor $T_{\mu\nu}$ is assembled with `assemble_rT` and the fluid frame quantities (J, H_i) (plus H^i) are computed using `calc_J_from_rT` and `calc_H_from_rT` (which are floored right after if necessary; Γ is also computed using `compute_Gamma`). The radiation source S_μ defined in Equ. (9) is calculated via `calc_rad_sources`. With that the matter-coupling sources are computed as

$$\mathbf{S} = \begin{pmatrix} \alpha \sqrt{\gamma}[\eta^0 - \kappa^0_a n] \\ -\alpha \sqrt{\gamma} S^\mu n_\mu \\ \alpha \sqrt{\gamma} S^\mu \gamma_{k\mu} \end{pmatrix}. \tag{41}$$

7. If the source update is implict, then we boost to the fluid frame and compute the fluid matter interaction and at the end boost back. These values are used for the implicit solve. In general we compute

$$E^* = E + \Delta t \mathbf{G}[E], \tag{42}$$

$$F^*_i = F_i + \Delta t \mathbf{G}[F_i], \tag{43}$$

$$N^* = N + \Delta t \mathbf{G}[N]. \tag{44}$$

The pack (E^*, F^*_i, N^*) is used to compute the closure $P_{\mu\nu}$, to assemble the stress-energy tensor $T_{\mu\nu}$, and the fluid frame quantities, i.e. (J^*, H^*_μ). With that we can estimate $T_{\mu\nu}$ and then boost back to the lab frame to get the new M1 variables (E, F_i) based on the updated stress-energy tensor. We then compute the source update wtih `source_update`, update the closure and compute the new radiation energy density in the fluid frame.

8. After the sources are computed we limit them using the specified limiters.

9. Lastly, we update the fields of the state vector $\mathbf{U}$.

radiation_m1_nurates.hpp: Defines the `NuratesParams` struct which is filled in the `radiation_m1.cpp` constructor if `BNS_nurates` is enabled. It also defines a function called `bns_nurates` which sets nurates specific things that are fed into `radiation_m1_calc_opacities_nurates.cpp`:

- Defines some unit conversions between code units and `BNS_nurates`.
- Fills variables for the four neutrino species with the inputs of η_0, η_1, $\kappa_{a,0}$, $\kappa_{a,1}$, $\kappa_{s,0}$ etc. Splits the thermal from the non-thermal contribution for $\kappa_{a,0/1}$ and $\eta_{0/1}$. The non-thermal parts from NEPS are currently being zerod right afterwards.
- The function also reads the baryon-number density and the temperature. If one of these is below `nb_min_fm-3` or `temp_min_mev`, then the emissivity/absorptivity/scattering coefficients are set to zero.
- The opacity parameter struct `GreyOpacityParams` from `BNS_nurates` is filled with the nurates parameters set in the M1 constructor on start-up, i.e. with all the flags for the corrections, reactions, etc. This struct also takes EOS quantities. If the reconstructed distribution is used, the fluid frame variables, i.e. (J, n, χ) and it then computes

the reconstructed distribution function with `CalculateDistrParamsFromM1` inside `BNS_nurates`. This function calls `CalculateThinParamsFromM1` and `CalculateThickParamsFromM1` using the input parameters and stores the output in a struct called `NuDistributionParams`. For the equilibrium distribution we call `NuEquilibriumParams` instead which is followed by a call to `ComputeM1DensitiesEq` that computes gray neutrino number and energy densities under the assumption of equilibrium. The eddington factor χ for all species is subsequently set to $\frac{1}{3}$.

- If the non-thermal separation is used between NEPS and the rest of the reactions, then we use `ComputeM1OpacitiesNonThermalSeparated` to treat NEPS separately from the thermal processes. As such NEPS is not included in the number emissivity η_0 and absorption $\kappa_{0,a}$. This function calls all the integrands/kernels that are specified by the user, i.e. for isoenergetic scattering off nucleons nurates calls `Scattering1DIntegrand` with `GaussLegendreIntegrate1DMatrix` and similar for beta-processes etc. This call returns the number emissivity and absorption coefficient, as well as energy emissivity, absorption, and scattering coefficients between thermal and non-thermal process and assigns them split onto the processes. If we are not using non-thermal separation, we then use `ComputeM1Opacities` which essentially does the same thing but does not split into thermal and non-thermal processes.
- Assert that the opacities are finite.
- Convert the opacities to code units.

The function `NeutrinoDens` computes the neutrino number and energy density with the approximate Fermi integrals in `radiation_m1_fermi.hpp`.
radiation_m1_calc_opacities_nurates.cpp: Defines a function called `CalcOpacityNurates` which wraps `CalcOpacityNurates_` based on the specified equation of state format. The wrapped function does the following:

- On the first specified warm-up cycles the equilibrium distribution is turned on, because the M1 moments are floored and a reconstucted distribution would yield garbage. This is only important of the reconstructed distribution is used.
- Collect the solution arrays, unit conversions etc.
- The Kokkos kernel that performs the opacity calculation first zeroes the number/energy emissivity/absorptivity coefficients as well as the energy scattering coefficient if we are inside an excision mask. Otherwise the spacetime metric $g_{\mu\nu}$ and $g^{\mu\nu}$ is calculated as well as the the volume factor $\sqrt{\gamma}$. Using helper functions to compute the Lorentz factor W, the four-velocity u^μ, the three-velocity v^i (as well as their contractions), and the fluid projector ${}^{\perp}h^\mu_\nu$ enables us to calculate the fluid frame M1 variables (J, H_i, n) from the Eulerian frame (E, F_i, N) using the closure $P_{\mu\nu}$. The computation proceeds as elsewhere in the code by computing the closure, assembling the stress-energy tensor and then projecting out the Eulerian variables into the fluid frame. Both J and n are undensitized afterwards.
- From the EOS the kernel fetches the baryon number density n_b, the pressure p, the electron fraction Y_e, the temperature T by inverting p (`GetTemperatureFromP`), the proton fraction Y_p with `GetProtonFraction`, the neutron fraction Y_n with `GetNeutronFraction`, the baryon chemical potential μ_b with `GetBaryonChemicalPotential`, the charge chemical potential μ_q with `GetChargeChemicalPotential`, and the electron-lepton chemical potential μ_{le} with `GetElectronLeptonChemicalPotential`. The neutron chemical potential μ_n is subsequently set to the baryon chemical potential μ_b, while the proton chemical potential is set to $\mu_p = \mu_b + \mu_q$, and lastly the electron chemical potential to $\mu_e = \mu_{le} + \mu_q$.
- The call to `bns_nurates` in `radiation_m1_nurates.hpp` gets the emissivities and opacities from `BNS_nurates` and stores the locally inside the kernel separated into energy and number, as well as thermal and non-thermal. If the non-thermal components is zero, then so are the opacities and emissivities. Asserting finiteness and non-negativeness is done afterwards for all components and for all species.
- If the kirchhoff law is used or the equilibrium distribution, then a new set of operations is performed: the effective optical depth $\tau = \min\left(\sqrt{\kappa_{a,\nu_e}\cdot(\kappa_{a,\nu_e}+\kappa_{s,\nu_e})}, \sqrt{\kappa_{a,\bar{\nu}_e}\cdot(\kappa_{a,\bar{\nu}_e}+\kappa_{s,\bar{\nu}_e})}\right)\Delta t$ is computed to decide, whether to compute the black body function for neutrinos assuming neutrino trapping or at fixed temperature and electron fraction. If τ is larger than the user set `opacity_tau_trap` then the neutrino black body function assuming trapped neutrinos is computed with `GetBetaEquilibriumTrapped` in `primitive-solver/eos.hpp` (if it fails the first time it is recomputed neglecting current neutrino data). Once weak equilibrium is found, the baryon/charge/electron-lepton chemical potentials are re-computed with the new charge fractions and then from there with `NeutrinoDens` the neutrino number and energy densities are calculated and asserted for finiteness. If we are not in a trapped regime, we

compute the black body function assuming a fixed temperature and electron fraction and call `NeutrinoDens` directly, which internally calls the approximate Fermi integrals. After that the opacities and emissivities are stored in the solution arrays and combine the optically thick and thin limits. In case of the equilibrium distribution we apply a correction factor for the absorption opacities for non-LTE effects to the electron-lepton neutrinos (this correction factor is applied to all species for the scattering coefficient). Also, in case of NEPS and equilibrium distribution we apply a correction to the NEPS emissivity because equilibrium over-populates the neutrino field in the decoupling region. If kirchhoff's law is enabled we apply the non-LTE correction only to the thermal part and keep the non-thermal NEPS number absorption as is.

radiation_m1_tasks.cpp: This sets the tasklist map which is called in `MeshBlockPack::AddPhysics()`. The tasks that are listed are:

- `InitRecv` (operator split before stagen): function to post non-blocking receives with MPI and initialize all boundary receive status flags to waiting
- `CopyCons` (operator split stagen): simply function that copies `u0` into `u1` in the first RK stage
- `SetMask` (operator split stagen): sets the radiation excision mask from the coordinate excision mask and zeroes the radiation fields in masked cells (runs each stage before the closure) - depends on `CopyCons`
- `FloorAndCalcClosure` (operator split stagen): computes χ with the function defined in `radiation_m1_calc_closure.cpp` - depends on the `SetMask` task
- `CalcOpacityNurates` (operator split stagen): computes the opacities and emissivities with `BNS_nurates` - depends on `FloorAndCalcClosure`
- `CalculateFluxes` (operator split stagen): calculates the fluxes $\mathbf{F}$ - depends on `CalcOpacityNurates`
- `SendFlux` (operator split stagen): pack/send restricted values of fluxes of conserved variables at fine/coarse boundaries - depends on `CalculateFluxes`
- `RecvFlux` (operator split stagen): receive/unpack restricted values of fluxes of conserved variables at fine/coarse boundaries - depends on `SendFlux`
- `TimeUpdate` (operator split stagen): performs the implicit time update to compute the new solution - depends on `RecvFlux`
- Post-processing: after the time update the usual operations such as restriction/prolongation and applying physical boundary conditions are performed.

Workflow

Here we summarize the process above for one cycle, and one cell:

1. The contribution of radiation to the stress-energy tensor is set via `SetTmunu` of the M1 scheme.
2. M1 operator split stage 1 ($\beta = 1/2$):
   (a) Copy solution array `u0` into intermediate stage array `u1` via `CopyCons`.
   (b) Set the radiation excision mask and ensure physicality via flooring.
   (c) Compute the closure χ from the root ξ of $f(\xi)$ with `FloorAndCalcClosure`.
   (d) `CalcOpacityNurates` takes the evolved M1 variables in the fluid frame, constructs the fluid frame variables via the radiation pressure tensor and the stress-energy tensor assembly and together with the equation of state vector and chemical potentials it is fed into `bns_nurates`. The latter calls `BNS_nurates` constructs the distribution function and by integration computes the opacities and emissivities, i.e. η_0, κ_0, η_1, $\kappa_{1,a}$, $\kappa_{1,s}$. In the case of the equilibrium function we apply correction factors to the opacities and emissivities depending on the regime we are in.
   (e) Calculate the fluxes $\mathbf{F}$ via `CalculateFluxes` using the linear combination of high- and low-order fluxes.
   (f) `TimeUpdate` implicitely solves for the solution without fluid backreaction.
3. M1 operator split stage 2 ($\beta = 1$):

(a) χ is recomputed from the solution of the previous step and the opacities are re-used.

(b) Update the solution with the solution of the implicit solve and turn on backreaction, i.e. add neutrino-matter coupling.

(c) Call the C2P inversion.

In one sentence: M1 evolves grey moments and therefore cannot know its own energy spectrum; `BNS_nurates` takes M1's three grey numbers per species (n_ν, J, χ) plus the fluid's thermodynamic state, rebuilds a spectrum consistent with them, integrates energy-dependent cross-sections over it, and returns the spectrum-averaged $(\eta, \kappa_a, \kappa_s)$ that close M1's source term $S^\mu = (\eta - \kappa_a J)u^\mu - (\kappa_a + \kappa_s)H^\mu$.

References

[1] David Radice, Sebastiano Bernuzzi, Albino Perego, and Roland Haas. A new moment-based general-relativistic neutrino-radiation transport code: Methods and first applications to neutron star mergers. *Mon. Not. Roy. Astron. Soc.*, 512(1):1499–1521, 2022.

[2] K. Takahashi, M. F. El Eid, and W. Hillebrandt. Beta transition rates in hot and dense matter, Jul 1978.